RESEARCH PAPER REVIEW

Boosting Faithful Multi-Modal LLMs via **Complementary Visual Grounding (CVG)**

IEEE Transactions on Image Processing, Vol. 34, 2025

Fu, Mao, Zhang, Zhang | University of Science and Technology of China

---

Document Contents

Part I: Full Paper Explanation | Part II: Detailed Improvement Work Plan

---

PART I: PAPER EXPLANATION

1. Core Problem Being Solved

Multimodal Large Language Models (MLLMs) — such as LLaVA, GPT-4V, and Qwen-VL — are designed to answer questions about images by jointly processing visual and textual information. Despite impressive performance across vision-language tasks, these models frequently produce hallucinations: generated text that is factually inconsistent with the actual visual content.

> *Example: Given an image of a dog sitting on a beach, the model might generate 'A dog is about to throw a stick to someone surfing' — fabricating a surfer that does not exist in the image.*

This hallucination problem has serious real-world consequences in domains such as:

- Medical imaging — incorrect diagnoses from fabricated findings
- Autonomous driving — unsafe decisions from hallucinated obstacles
- Security systems — false threat assessments
- Scientific analysis — corrupted results from invented data

The central goal of this paper is to make MLLMs generate responses that are truthfully grounded in the actual image content, without requiring additional training data, external models, or post-hoc correction pipelines.

2. Root Causes of Hallucination

A key contribution of the paper is identifying and empirically validating two fundamental causes of hallucination, rather than simply treating surface symptoms.

2.1 Insufficient Visual Context

Model doesn't clearly capture important regions
So it guesses from language memory

MLLMs dynamically focus their visual attention on query-relevant image regions. When those specific regions are ambiguous, incomplete, or not clearly encoded, the model cannot ground its answer in visual evidence. Instead, it falls back on its internal language knowledge and statistical patterns learned during pretraining — producing plausible-sounding but visually unsupported content.

The paper validates this experimentally: masking query-relevant image regions causes significantly higher hallucination rates than random masking, proving that visual completeness is critical to faithful generation.

2.2 Progressive Textual Drift

During long-form response generation, the model's attention to the original image progressively weakens. As more text tokens are generated, the model increasingly attends to its own previously generated words rather than the image, drifting away from visual grounding toward its linguistic prior.

The paper validates this with quantitative analysis on the CHAIR benchmark: hallucination rates rise monotonically with response length, confirming that the drift from visual grounding is the mechanism driving this pattern.

3. MLLM Architecture Background

To understand CVG, it is important to understand how MLLMs normally work:

Component	Role
Vision Encoder (ViT)	Splits the image into N patches and encodes them as visual token embeddings
Projector (MLP)	Maps visual embeddings into the language model's embedding space
Language Decoder (LLM)	Generates the response token-by-token using self-attention over visual + text tokens

Generation is autoregressive: each token is predicted as P(next token | image tokens, query tokens, previous response tokens). The key problem is that as generation proceeds, the image influence diminishes and language pattern influence grows, causing hallucinations.

4. CVG Framework Overview

The paper proposes Complementary Visual Grounding (CVG) — a training-free decoding framework that addresses both root causes by leveraging only the MLLM's intrinsic architecture. CVG requires no external tools, models, retraining, or additional data.

CVG comprises three core modules working in sequence:

Module	Purpose
1. Visual Token Disentanglement	Separates image patches into two complementary branches: chosen (relevant) and rejected (irrelevant)
2. Dynamic Attention Calibration	Adjusts attention weights during generation to maintain visual grounding and prevent textual drift
3. Grounding Flow Fusion	Combines outputs of both branches using adaptive contrastive decoding to produce a faithful response

> *Core Intuition: Like two detectives — one relying on real evidence (chosen branch) and one relying on guesswork (rejected branch). When they disagree, trust the evidence. This contrastive signal suppresses hallucinated tokens.*

5. Module 1 — Visual Token Disentanglement

This module identifies which image patches are most relevant to the user's query, then creates two separate evidence branches for parallel processing.

5.1 Computing Token Significance

A hybrid significance score is computed for each visual token using two complementary signals:

Signal 1: Query-to-Image Cross-Attention

Measures how strongly each image patch is attended to by the query tokens. High attention = strong semantic relationship between query and that image region.

Mathematically: A_Q2I = softmax(Q_q * K_v^T / sqrt(d))

The attention from the last query token (which captures overall query intent) is averaged across intermediate transformer layers and attention heads to produce a per-visual-token relevance score: a_cross.

Signal 2: Image Self-Attention (Query-Agnostic Saliency)

Uses the [CLS] token's attention within the Vision Transformer to capture intrinsically important image regions — those that are visually salient regardless of the query. This ensures important regions not explicitly mentioned in the query are not overlooked.

Mathematically: A_I2I = softmax(Q_v * K_v^T / sqrt(d))

The [CLS]-to-all-patches attention is averaged across intermediate ViT layers and heads to produce: a_self.

Combined Score

The final importance score is: a = a_cross + a_self

This gives each visual token an overall significance value that captures both query relevance and intrinsic visual saliency.

5.2 Partitioning into Complementary Branches

Using the significance scores and a hyperparameter alpha (selection ratio, default 0.7):

- Chosen Set: Top alpha*N visual tokens with highest scores — high-fidelity, low-uncertainty evidence stream
- Rejected Set: Remaining (1-alpha)*N tokens — low-fidelity, high-uncertainty background/irrelevant content

Crucially, partitioning is done at the raw image patch level before the vision encoder. Each branch is independently processed by the ViT and MLP projector, preventing information leakage between branches.

6. Module 2 — Dynamic Attention Calibration

This module counteracts progressive textual drift by dynamically adjusting attention weights during generation, re-anchoring the model to visual evidence as the response grows longer.

6.1 Calibration Mechanism

During generation of the i-th response token, a calibration value delta_i is added to visual token attention scores and subtracted from textual token scores (or vice versa):

delta_i = beta * (1 - exp(-i / tau))

Parameter	Meaning
beta	Controls maximum calibration strength (default 0.2) — prevents over-disrupting language capabilities
tau	Controls the growth rate of calibration (default 10) — calibration builds gradually over generation steps
i	Current generation step — calibration intensifies as response length grows

6.2 Branch-Specific Application

The calibration is applied oppositely to each branch, creating divergent generation pathways:

- Chosen Branch: Visual token attention scores += delta_i; Textual token attention scores -= delta_i (enforces visual grounding)
- Rejected Branch: Visual token attention scores -= delta_i; Textual token attention scores += delta_i (forces language-prior reliance, maximizing hallucination signal)

This dual strategy maximizes the contrast between the two branches, giving the fusion module a powerful signal to distinguish faithful from hallucinatory content.

7. Module 3 — Grounding Flow Fusion

This module integrates the output distributions of both branches to produce the final, faithfully grounded token prediction.

7.1 Plausibility Filtering

A candidate vocabulary set V_c is constructed from the chosen branch's output distribution. Only tokens with probability >= eta * max_probability (eta = 0.2) are considered, filtering out implausible predictions and improving efficiency.

7.2 Adaptive Contrastive Decoding

The KL divergence D_KL(p_chosen || p_rejected) between the two branch distributions is measured to determine whether contrastive correction is necessary:

Case	Action

Divergent Branches (KL >= epsilon)	Branches disagree — likely hallucination present. Apply contrastive decoding: logit_final = logit_chosen + gamma*(logit_chosen - logit_rejected). This amplifies the chosen signal while penalizing hallucinated content.
Convergent Branches (KL < epsilon)	Branches agree — visual evidence is unambiguous. Use chosen branch output directly: logit_final = logit_chosen. No penalty needed.

This adaptive strategy ensures hallucinations are suppressed only when genuinely contested, preserving generation quality in unambiguous cases.

Hyperparameters: gamma = 1 (contrastive strength), epsilon = 0.01 (KL threshold).

8. Experimental Results

8.1 Hallucination Mitigation Benchmarks

Benchmark	Description & CVG Achievement
POPE	Object existence questions (random/popular/adversarial splits). CVG achieves new state-of-the-art accuracy across all splits and all model scales.
CHAIR	Image captioning hallucination metric (instance-level and sentence-level). CVG significantly reduces hallucinated objects in generated captions.
AMBER	Multi-dimensional LLM-free benchmark for both generative and discriminative tasks. CVG outperforms all competing methods.
MME-Hal	Object existence, count, position, and color evaluation. CVG consistently improves attribute-level and object-level accuracy.
MM-Hal / HallusionBench	Model-based evaluation metrics (GPT-4 judged). CVG achieves best performance across all tested architectures.

8.2 General Capability Benchmarks

CVG not only reduces hallucinations but also improves or maintains general performance, unlike many hallucination mitigation methods that trade off general ability:

- Task-oriented VQA (VQA-v2, GQA, ScienceQA): Consistent improvement from better visual grounding
- Instruction-following (MM-Bench, MM-Vet, LLaVA-Wild): Significant gains — CVG shows strong generalization
- Limitation: Slight degradation on TextVQA (OCR tasks) — fine-grained text patches may be misclassified as rejected tokens

8.3 Model Generalization

CVG was tested on multiple model families and scales:

- LLaVA-1.5 (7B and 13B)

- LLaVA-Next / LLaVA-1.6 (7B and 13B)
- Qwen2-VL and Qwen2.5-VL
- InternVL-2.5 (1B to 38B) — scaling law analysis confirms consistent improvement as model size grows

8.4 Inference Efficiency

Despite running two parallel branches, CVG maintains competitive efficiency because each branch processes only half the visual tokens. The initial attention-score computation is done once and shared. FLOPs analysis shows CVG has lower computational cost than competing methods like VCD.

9. Key Contributions Summary

1. Systematic identification and empirical validation of two root causes of MLLM hallucination: Insufficient Visual Context and Progressive Textual Drift — shifting focus from symptom treatment to mechanism addressing.
2. Proposal of the CVG framework: a training-free, model-agnostic approach that requires no external tools, post-hoc correction, or additional training data.
3. Design of three synergistic modules: Visual Token Disentanglement, Dynamic Attention Calibration, and Grounding Flow Fusion — operating at input, processing, and output stages respectively.
4. State-of-the-art performance across multiple hallucination and general-purpose benchmarks, validated across diverse architectures and parameter scales.

10. Limitations

- Text-rich images: Attention-based token selection may classify fine-grained text patches as rejected, degrading OCR task performance.
- Heuristic layer selection: Token selection relies on averaged attention across predefined layers, which may not be optimal for all image types or complex reasoning tasks.
- Dense/crowded scenes: In images with many closely packed objects, the attention mechanism may fail to prioritize all significant visual patches.

PART II: DETAILED IMPROVEMENT WORK PLAN

1. Objective and Motivation

The CVG framework demonstrates strong results in hallucination mitigation. However, five specific limitations create clear opportunities for principled, high-impact improvements:

Limitation	Improvement Opportunity
Attention-only token scoring misses fine-grained/text regions	Multi-factor adaptive token scoring with objectness and spatial signals
No spatial relational reasoning between objects	Graph-based spatial grounding module
Dual-branch inference increases compute cost	Single-pass efficient approximation of rejected branch
No semantic verification of generated text vs. image	Cross-modal consistency checking with CLIP
Visual grounding may still drift in very long responses	Visual memory bank for persistent grounding across generation

2. Improvement 1: Adaptive Multi-Factor Visual Token Scoring

Problem with Current Approach

The current token scoring formula a = a_cross + a_self relies purely on attention-based signals. Attention scores can be noisy: small but critical objects may receive low attention, and fine-grained text characters are consistently misclassified as irrelevant.

Proposed Solution

Replace the binary attention sum with a learned multi-factor scoring function:

token_score = w1 * a_cross + w2 * a_self + w3 * spatial_saliency + w4 * objectness_score

Factor	Source
a_cross	Query-to-image cross-attention
a_self	ViT [CLS] self-attention
spatial_saliency	Gradient-based saliency or GradCAM
objectness_score	Lightweight detector (DETR/YOLO)

Step-by-Step Implementation

5. Run a lightweight object detector (DETR-nano or YOLOv8-n) on the input image to obtain bounding boxes and confidence scores per detected object.
6. Map detected bounding boxes to the ViT patch grid: assign each patch the maximum objectness score from overlapping detections.
7. Compute gradient-based spatial saliency using GradCAM on the ViT's final layer with respect to the query.
8. Normalize all four scores to [0,1] and combine using weights w1=0.3, w2=0.2, w3=0.25, w4=0.25 (tunable via ablation).
9. Use the combined score in place of the original 'a' for chosen/rejected partitioning.

Justification

- Object detectors are trained explicitly to identify complete objects — they naturally handle small objects that attention may miss.
- Gradient-based saliency provides complementary signal not captured by attention rollout.
- This directly addresses the TextVQA degradation by assigning high objectness scores to text bounding boxes.

3. Improvement 2: Spatially-Aware Graph Grounding

Problem with Current Approach

CVG performs token-level disentanglement but entirely ignores spatial relationships between objects. Questions like 'What is the dog next to?' or 'Is the cup on the left or right of the plate?' require reasoning about relative positions — information not encoded in individual patch embeddings.

Proposed Solution

Construct an object-level spatial relation graph and inject its features into the chosen branch's visual tokens before generation.

Step-by-Step Implementation

10. Detect objects and extract region-level features using RoI Align from the object detector's feature pyramid.
11. Construct a spatial graph: nodes = detected objects (region features), edges = spatial relations (above/below/left-of/right-of/inside/contains) computed from bounding box geometry.
12. Encode the graph using a lightweight Graph Attention Network (GAT) with 2 layers to produce relation-aware node embeddings.

13. Use cross-attention between GAT node embeddings and the chosen visual tokens to inject spatial relational information.
14. The enhanced chosen visual tokens replace the original chosen tokens in the CVG pipeline.

Justification

- Spatial relationships are a documented cause of hallucinated descriptions ('the cat is to the left of the dog' when it is actually to the right).
- Graph Neural Networks are specifically designed to model relational structures — they are the natural fit for this problem.
- This improvement is complementary to token disentanglement: it enriches the chosen branch rather than changing its selection.

4. Improvement 3: Visual Memory Bank for Long-Response Grounding

Problem with Current Approach

Dynamic Attention Calibration counteracts textual drift via attention score adjustment, but it cannot directly inject visual information into the model's computation. For very long responses (256+ tokens), the calibration signal alone may be insufficient to maintain strong grounding.

Proposed Solution

Introduce a visual memory bank that stores key chosen visual embeddings and is queried at each generation step to provide explicit visual anchors.

Step-by-Step Implementation

15. Before generation begins, extract and store the top-K (K=20) most important chosen visual token embeddings in a memory buffer M.
16. At each generation step i, compute cosine similarity between the current query vector and all memory embeddings: sim(q_r, m_k) for k in M.
17. Retrieve the top-3 most similar memory embeddings and compute their attention-weighted sum as a visual context vector c_i.
18. Inject c_i into the generation by adding it as an additional visual token in the attention computation of the chosen branch (gated by a learnable scalar to control injection strength).
19. This creates a persistent visual anchor that supplements attention calibration, especially for later generation steps where calibration alone weakens.

Justification

- Memory retrieval provides direct visual evidence injection rather than indirect attention manipulation.
- The cosine similarity retrieval ensures that only contextually relevant visual embeddings are retrieved, not all memory contents.
- This is training-free: no parameters are learned; the gating scalar can be fixed to 0.1 based on validation.

5. Improvement 4: Efficient Single-Pass Rejected Branch Approximation

Problem with Current Approach

CVG requires two separate forward passes through the LLM — one for the chosen branch and one for the rejected branch. This approximately doubles inference time and memory consumption, which limits deployment feasibility.

Proposed Solution

Approximate the rejected branch distribution using an attention masking simulation within a single forward pass, eliminating the second full LLM inference.

Step-by-Step Implementation

20. Perform a single forward pass with the chosen branch through the full LLM.
21. Cache all intermediate attention maps and hidden states from the chosen branch pass.
22. Simulate the rejected branch by applying a post-hoc attention re-weighting to the cached states: set visual token attention weights to near-zero and boost textual token weights, matching what a rejected-branch calibration would produce.
23. Compute the approximate rejected distribution from the re-weighted hidden states using only the final LM head computation (no additional transformer blocks needed).
24. Use this approximate distribution for the KL divergence check and contrastive decoding in GFF.

Justification

- The key insight: the rejected branch's purpose is to represent a linguistically-biased distribution. This can be approximated by suppressing visual attention in the existing computation rather than re-running all transformer layers.
- Estimated speedup: 40-50% reduction in inference time by eliminating the second full forward pass.
- Maintains the contrastive signal quality since the approximation captures the same linguistic-prior drift that the rejected branch is designed to expose.

6. Improvement 5: Cross-Modal Semantic Consistency Verification

Problem with Current Approach

CVG's Grounding Flow Fusion compares token-level probability distributions between branches but does not verify whether the semantic content of the generated response is consistent with the image at a higher, sentence-level.

Proposed Solution

Add a lightweight cross-modal semantic consistency checker using a pre-trained CLIP model to verify generated content against the image.

Step-by-Step Implementation

25. During generation, maintain a rolling sentence buffer that collects generated tokens until a sentence boundary (period, comma, end-of-sequence) is detected.
26. When a sentence boundary is reached, encode the completed sentence using CLIP's text encoder to obtain a text embedding t_s.
27. Compare t_s against the image embedding (pre-computed once before generation) using cosine similarity: score = cos(t_s, img_emb).
28. If the score falls below a threshold (e.g., 0.25), apply a penalty to the logits of the next token prediction to discourage continuation of the low-similarity semantic direction.
29. High-similarity sentences proceed without penalty, ensuring the check does not degrade quality for faithful content.

Justification

- CLIP embeddings encode holistic semantic meaning — they can detect when a sentence describes content entirely absent from the image.
- Sentence-level checking complements token-level contrastive decoding, catching hallucinations that emerge from cumulative drift rather than individual token errors.
- This adds only one CLIP text encoding per sentence, which is extremely lightweight compared to LLM generation cost.

7. Evaluation Plan

7.1 Hallucination Benchmarks

Benchmark	Metric
POPE (random/popular/adversarial)	Accuracy

CHAIR (instance + sentence level)	Hallucination rate (lower is better)
AMBER (generative + discriminative)	CHAIR, Cover, Hal, Cog, Accuracy
MME-Hal	Accuracy on object/attribute questions
HallusionBench	Model-based GPT-4 evaluation score

7.2 General Capability Benchmarks

Benchmark	Focus
VQA-v2 / GQA / ScienceQA	Visual perception and scientific reasoning
TextVQA	OCR and text reading (specific regression target for Improvement 1)
MM-Bench / MM-Vet	Complex instruction-following
LLaVA-Wild	Diverse real-world scenarios

7.3 Efficiency Analysis

- FLOPs comparison: CVG baseline vs. CVG + Efficient Approximation (Improvement 4)
- Inference latency: measured on A100 GPU across response lengths 64/128/256/512
- GPU memory: peak memory consumption per batch

8. Ablation Studies

Study	Comparison
Token Scoring Strategy	Attention-only vs. Multi-factor (w/ objectness) vs. Multi-factor (w/ saliency) vs. Full multi-factor
Spatial Graph Module	CVG baseline vs. CVG + GAT graph grounding vs. CVG + MLP region features (no graph)
Visual Memory Bank	CVG baseline vs. CVG + memory (short responses) vs. CVG + memory (long responses 256+)
Efficient Approximation	Dual full-pass vs. Single-pass approximation — comparing hallucination rate and inference time
Consistency Verification	Without CLIP check vs. With CLIP check — measuring false positive rate and hallucination correction rate

9. Project Timeline

Phase & Duration	Tasks
Phase 1 — Weeks 1-2: Baseline Reproduction	Set up LLaVA-1.5 (7B/13B) + CVG pipeline in PyTorch. Reproduce all baseline numbers on POPE, CHAIR, AMBER.
Phase 2 — Weeks 3-4: Multi-Factor Token Scoring	Integrate DETR-nano + GradCAM. Implement weighted scoring. Ablate all four factors. Target: TextVQA improvement.
Phase 3 — Weeks 5-6: Spatial Graph Grounding	Build object detection + GAT graph pipeline. Integrate into chosen branch. Evaluate on spatial reasoning queries.
Phase 4 — Weeks 7-8: Visual Memory Bank	Implement memory buffer with cosine retrieval. Integrate gated injection. Evaluate specifically on 256+ token responses.
Phase 5 — Week 9: Efficient Approximation	Implement single-pass rejected branch simulation. Measure FLOPs and latency. Verify hallucination performance parity.
Phase 6 — Week 10: Cross-Modal Consistency	Integrate CLIP sentence-level checker. Tune similarity threshold on validation set. Evaluate false-positive rate.
Phase 7 — Weeks 11-12: Full Evaluation & Writeup	Run all benchmarks on full improved system. Conduct all ablation studies. Prepare final results and discussion.

10. Key Discussion Points for the Meeting

Be prepared to address the following questions:

30. Why is attention alone insufficient for visual token grounding? What does objectness detection add that attention cannot provide?
31. How does the spatial graph module avoid adding excessive computational overhead at inference time?
32. What is the theoretical justification for approximating the rejected branch via attention re-weighting rather than a full second pass?
33. How do you prevent the CLIP consistency checker from penalizing correct, highly detailed descriptions that naturally have lower CLIP similarity?
34. What is the expected trade-off between hallucination reduction and general VQA performance for each improvement?
35. How does your work extend beyond decoding-time fixes? Could these insights inform future MLLM training procedures?

11. Expected Outcomes Summary

Improvement	Primary Expected Gain
Multi-Factor Token Scoring	Fixes TextVQA regression; improves dense/crowded scene handling
Spatial Graph Grounding	Reduces spatial relationship hallucinations; improves GQA compositional reasoning

Visual Memory Bank	Reduces hallucination rate in long responses (256+ tokens); improves AMBER generative scores
Efficient Approximation	40-50% inference speedup; enables practical deployment without sacrificing hallucination mitigation
Cross-Modal Consistency	Catches sentence-level semantic drift; complements token-level contrastive decoding

---

End of Document